Fake News Detection on the WELFake Dataset: A Classical Machine Learning Pipeline

Vu Ngoc Mai Nguyen
Machine Learning Project
Digital Transformation Management
Email: vungocngocmai.nguyen@studio.unibo.it

***Abstract*—This mini paper presents a fake news detection workflow based on the WELFake dataset. The task is formulated as a supervised binary classification problem in which each article is assigned to either the fake or the real class. The project follows a CRISP-DM style methodology: the raw data are inspected, missing and duplicate observations are handled, textual and stylistic features are engineered, and three classical machine learning models are compared. The final pipeline combines TF-IDF features extracted from the concatenation of title and article body with numeric indicators such as text length and capitalization ratio. A Random Forest classifier is used as the main model because it provides strong predictive performance while retaining a useful level of interpretability through feature importance analysis. A k-nearest neighbors model is also evaluated as an additional comparison. The experimental results show that Random Forest obtains a stronger error profile than k-NN, with fewer false positives and fewer false negatives on the held-out test set. The paper also discusses the role of data quality, leakage prevention, and interpretability in the design of a reliable fake news detection system.**

***Index Terms*—fake news detection, WELFake, TF-IDF, Random Forest, k-nearest neighbors, text classification, CRISP-DM**

I. Introduction

Fake news detection is the task of automatically distinguishing fabricated or unreliable news articles from trustworthy ones. The problem is relevant because online articles can be shared very quickly, often before a human fact-checking process has time to intervene. A machine learning model cannot decide truth by itself, but it can provide a first risk signal and help prioritize content that should be reviewed by analysts or moderators.

In this project, fake news detection is treated as a binary supervised learning problem. Each observation contains a title, a full article text, and a label. The goal is to learn a function that maps article content and simple stylistic indicators to one of two classes: fake or real. The approach is intentionally based on classical machine learning rather than large language models. This makes the workflow easier to reproduce, inspect, and explain in an academic setting.

The complete workflow follows the CRISP-DM logic. The business understanding phase defines the operational purpose of the model. The data understanding phase checks the structure of the dataset, the distribution of the labels, and the behavior of text length across classes. The data preparation phase creates a combined text field and transforms the corpus with TF-IDF. The modeling phase compares a Decision Tree baseline, a Random Forest classifier, and a k-nearest neighbors classifier. The evaluation phase uses confusion matrices and classification metrics to compare the resulting error profiles.

The contribution of this mini paper is a concise description of the complete pipeline and of the most important experimental findings. Particular attention is given to four graphical outputs from the notebook: the class and length exploration, the Random Forest confusion matrix, the k-NN confusion matrix, and the Random Forest feature importance plot. These figures connect the data understanding phase with the final model interpretation.

II. Dataset and Problem Definition

The dataset used in the project is WELFake, a text classification dataset for fake news detection. Each record includes a news title, the article body, and a binary target label. In the notebook convention, fake and real articles are represented as two separate classes. Before modeling, title and text are concatenated into a single field so that the classifier can exploit both headline-level and body-level information.

A. Business Objective

The practical objective is to build a model that can flag suspicious news articles before they are manually reviewed. In a realistic deployment, the model would not remove or censor content automatically. Instead, it would assign a risk score or a predicted class. Articles predicted as fake could then be placed in a priority queue for additional verification.

This framing is important because the cost of the two error types is different. A false negative means that a fake article is classified as real and may remain undetected. A false positive means that a real article is incorrectly flagged as fake. Both errors matter: false negatives reduce the safety of the system, while false positives can damage trust and create unnecessary moderation work.

B. Data Mining Objective

The data mining objective is to train a classifier with good generalization performance on unseen examples. This requires a clean split between training and test data. Text vectorization and numeric scaling must be fitted only on the training set, then applied to the test set. Duplicate removal is also important because duplicated articles can otherwise

appear in both partitions and inflate the apparent performance of the model.

III. Data Understanding

The data understanding phase focuses on the relationship between the label and basic article characteristics. Fig. 1 shows two exploratory plots. The first plot reports the class distribution of the working dataset. The two classes are close enough to make the classification task meaningful without severe imbalance. The second plot compares the combined text length by class. Fake articles show a slightly higher median and a wider interquartile range, while real articles include several high-length outliers.

The length distribution is useful because it shows that article size may contain some predictive information. However, length alone cannot be used as the main decision rule. A real article may be short, a fake article may be long, and many observations overlap. For this reason, the project combines lexical content with numeric stylistic variables instead of relying on a single descriptive statistic.

The exploratory phase also motivates the use of TF-IDF. News articles are high-dimensional textual objects: each article contains many tokens, and the importance of a word depends on how frequent it is in the specific article and across the whole corpus. TF-IDF reduces the weight of very common words and highlights terms that are more discriminative. This makes it a suitable representation for classical machine learning models.

IV. Data Preparation

Data preparation converts the raw table into a modeling-ready matrix. First, missing title and text values are handled so that the concatenation step does not fail. Second, articles with unusable textual content are removed. Third, duplicate rows are dropped before the train-test split. This order prevents information leakage, because the same article should not be available both during training and during evaluation.

A. Text Construction

The main textual input is a variable called `combined_text`, obtained by joining title and body. This choice is appropriate because the title often contains compact and highly informative wording, while the body contains context, entities, and repeated expressions that can help the classifier. Using only the title would discard most of the available text; using only the body would ignore headline style.

B. Numeric Features

In addition to TF-IDF features, the notebook builds a small set of numeric variables. These include title length, text length, combined text length, and capitalization ratios. Such features are not meant to determine credibility by themselves. Instead, they complement the TF-IDF representation with stylistic signals. For example, exaggerated capitalization may be correlated with attention-seeking headlines, but it is not a definitive indicator of misinformation.

TABLE I
Main feature groups used in the pipeline.

Feature group	Description
TF-IDF text features	Sparse lexical representation of `combined_text`
Length features	Word counts for title, text, and combined article
Capitalization features	Ratios of uppercase alphabetic characters
Target label	Binary class: fake or real article

C. Vectorization and Scaling

The text is transformed with a TF-IDF vectorizer. The vectorizer is fitted on the training corpus only and then reused on the test corpus. This prevents information from the test set from influencing the vocabulary and inverse document frequency values. Numeric features are standardized with the same principle: the scaler learns means and standard deviations from the training data, then applies the learned transformation to the test data.

The final feature matrix is obtained by concatenating the sparse TF-IDF matrix and the dense standardized numeric matrix. This representation allows classical models to use both lexical and structural information.

V. Modeling Approach

The project compares three classical supervised algorithms. The first is a Decision Tree classifier, used as an interpretable baseline. A single tree is easy to explain because it creates a sequence of rules, but it can overfit sparse high-dimensional text data. The second is a Random Forest classifier, selected as the main model. Random Forest averages many trees and usually reduces the instability of a single tree. The third is k-nearest neighbors, included as an additional comparison model based on similarity between articles.

A. Random Forest

Random Forest is well suited to this project because it can handle nonlinear interactions between terms and numeric indicators. It also provides feature importance values, which are useful for interpreting the model at a global level. In text classification, feature importance should be interpreted carefully: an important word is a strong model signal in this dataset, not proof that the word is inherently fake or real. Nevertheless, the plot helps verify whether the model is using meaningful textual and stylistic cues rather than only one accidental variable.

B. k-Nearest Neighbors

k-NN classifies a new article by comparing it with similar articles in the training set. This idea is intuitive for text classification: articles that use similar vocabulary may belong to the same class. However, k-NN is less attractive for large sparse datasets because prediction requires distance computations against many training examples. It is therefore useful as a comparison but less suitable as the main production model.

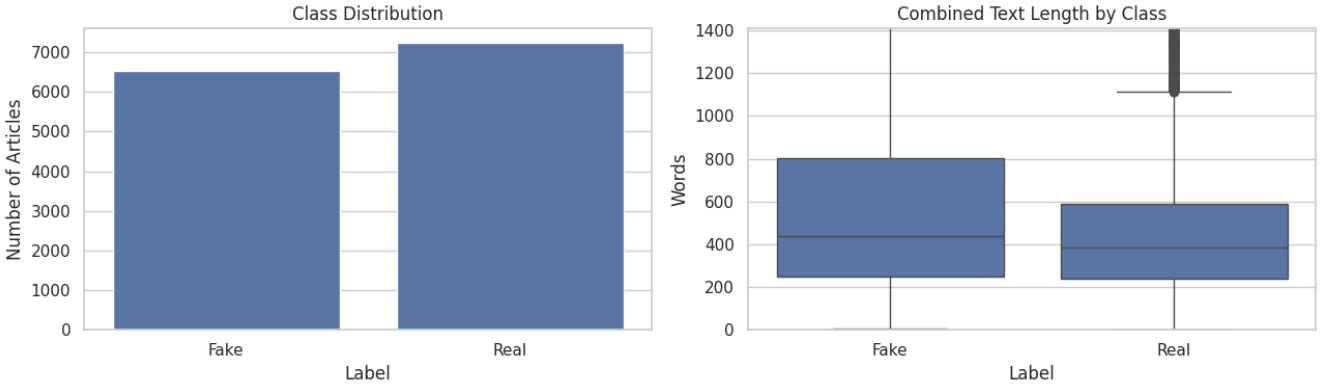

Fig. 1. Exploratory analysis of the working dataset. The left chart reports the class distribution, while the right chart compares the combined text length by class.

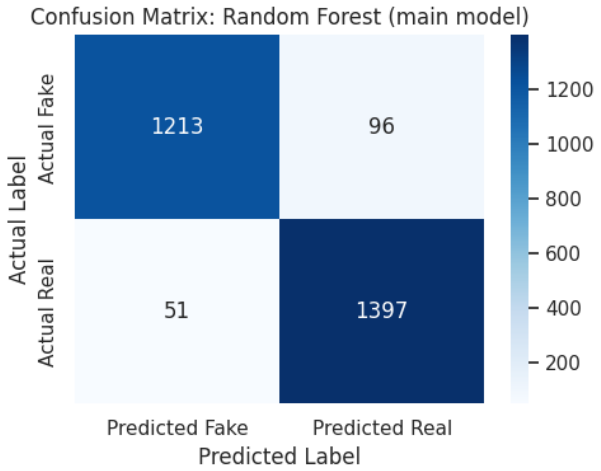

Fig. 2. Confusion matrix for the Random Forest main model. The model produces a low number of false positives and false negatives on the test set.

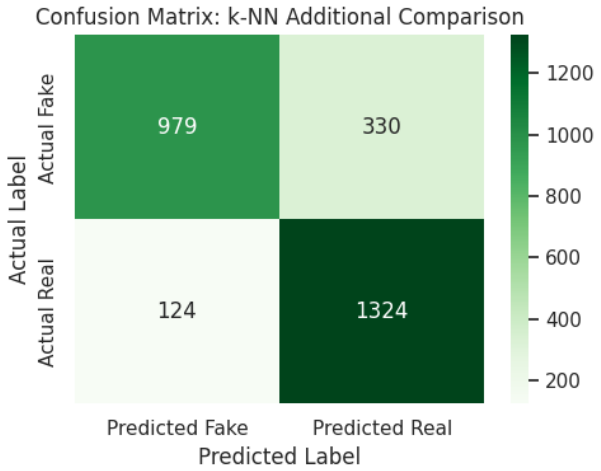

Fig. 3. Confusion matrix for the k-NN additional comparison model. Compared with Random Forest, k-NN makes more errors, especially by predicting fake articles as real.

VI. Evaluation

The evaluation is based on a held-out test set. For each classifier, the notebook computes predictions and compares them with the true labels. Confusion matrices are central because they show not only the number of correct predictions, but also the specific type of mistakes made by the model.

A. Random Forest Test Results

Fig. 2 shows the confusion matrix for the Random Forest model. The model correctly predicts 1213 fake articles and 1397 real articles. It misclassifies 96 fake articles as real and 51 real articles as fake. The resulting accuracy is approximately 94.7%. The matrix indicates that Random Forest performs strongly on both classes and keeps both types of errors relatively low.

Using the real class as the positive class, the Random Forest reaches a precision of about 93.6% and a recall of about 96.5%. Using the fake class as the reference class, the recall is about 92.7%. These values suggest a balanced model: it is not simply biased toward one label, and it recovers most examples from both classes.

B. k-NN Additional Comparison

Fig. 3 reports the confusion matrix for the k-NN comparison model. It correctly predicts 979 fake articles and 1324 real articles. However, it misclassifies 330 fake articles as real and 124 real articles as fake. The accuracy is approximately 83.5%, which is substantially lower than the Random Forest result.

The most important difference is the number of fake articles predicted as real. In a fake news detection scenario, this error can be particularly costly because it means that suspicious content is missed. The k-NN model still recognizes many real articles correctly, but its error profile is weaker for the moderation-oriented objective of the project.

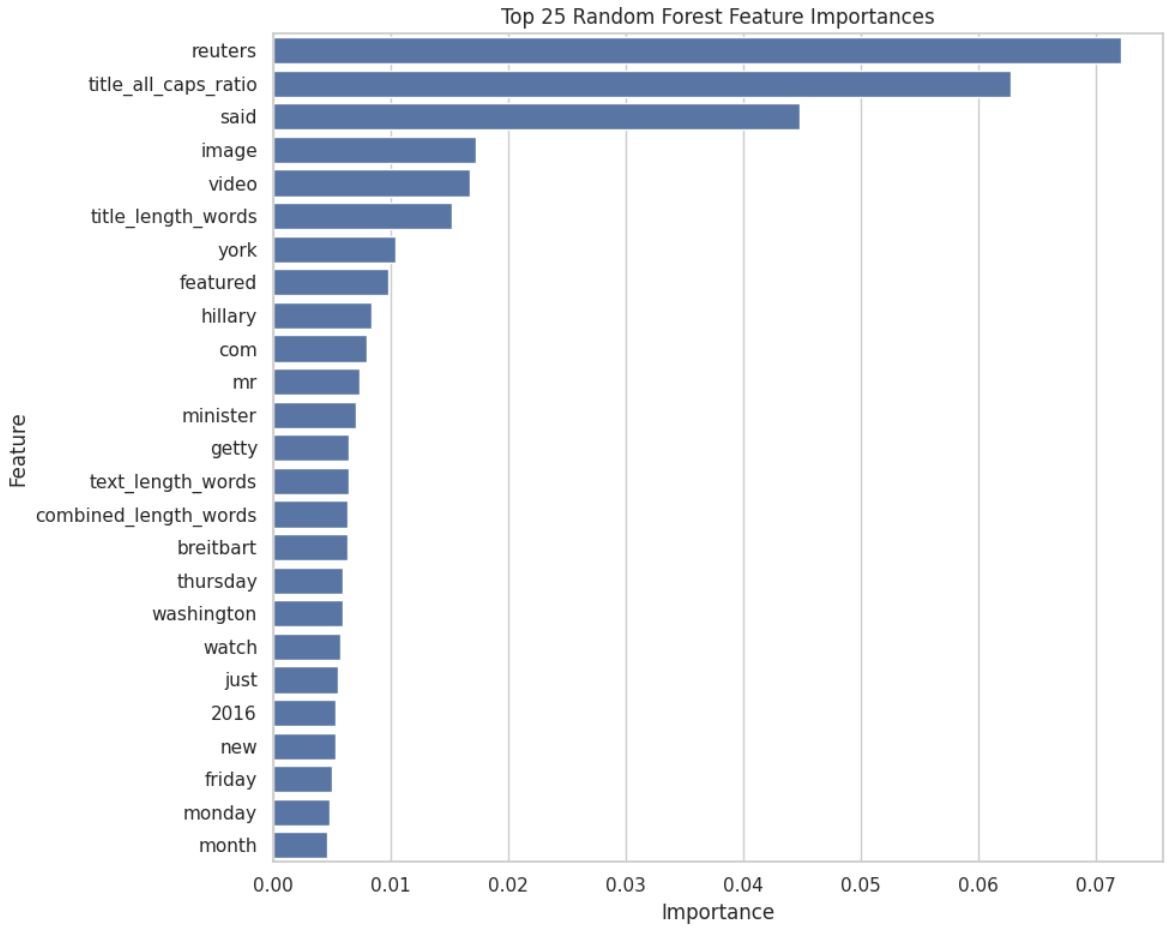

Fig. 4. Top 25 Random Forest feature importances. The model relies on both lexical indicators and engineered stylistic features.

C. Feature Importance

Fig. 4 reports the top 25 Random Forest feature importances. The most influential features include terms such as `reuters` and `said`, together with stylistic variables such as `title_all_caps_ratio` and `title_length_words`. This confirms that the classifier uses a mixture of lexical and engineered features.

The high importance of source-like or agency-related terms should be interpreted with caution. These words may reflect dataset-specific regularities, such as the writing style of particular sources, rather than universal markers of truthfulness. This is a common limitation in fake news detection: the model may learn correlations from the training corpus that do not transfer perfectly to future data or to different news domains.

VII. Discussion

The comparison between Random Forest and k-NN supports the choice of Random Forest as the main model. Both algorithms use the same prepared representation, but Random Forest achieves a better test error profile. The difference is especially visible in the fake-to-real mistakes: Random Forest misses far fewer fake articles than k-NN. This matters because one of the primary goals of a fake news detection system is to reduce the amount of harmful content that passes through the filter.

The exploratory plots also help explain the feature design. The class distribution indicates that the dataset does not suffer from an extreme imbalance problem, so the model is not forced to learn only a majority-class rule. The text length plot shows that the two classes have overlapping but not identical length distributions. This justifies including length variables, while also confirming that lexical TF-IDF features are necessary.

The feature importance plot provides a useful diagnostic view of the trained Random Forest. The model does not rely exclusively on article length or capitalization; it also uses words and tokens extracted from the text. At the same time, some of the most important terms may be connected to source identity or writing conventions. Therefore, feature importance should be used for model auditing, not for making causal claims about misinformation.

VIII. Threats to Validity

Several threats to validity should be considered. The first is dataset bias. WELFake contains articles from specific sources and time periods. A classifier trained on this dataset may learn patterns that are highly predictive inside the dataset but

less effective on new sources. External validation on another fake news corpus would be necessary before claiming broader generalization.

The second threat is label quality. The supervised approach assumes that the labels are correct, but credibility labels can be noisy or ambiguous. Some articles may mix true and false claims, and some may be labeled according to source reputation rather than detailed factual verification. Label noise can limit the maximum performance of any supervised model.

The third threat is temporal drift. The language of misinformation changes over time. Topics, entities, and rhetorical strategies evolve, so a static classifier may become outdated. In a real deployment, the model would require periodic retraining and continuous monitoring.

Finally, there is an ethical limitation. A false positive can unfairly flag legitimate journalism, while a false negative can let misinformation spread. For this reason, the model should be used as a decision-support tool rather than as an autonomous truth judge. Human review, transparency, and appeal mechanisms are essential in any real application.

IX. Conclusion

This paper presented a classical machine learning pipeline for fake news detection on the WELFake dataset. The workflow combines TF-IDF features from title and article body with numeric stylistic indicators. The exploratory analysis shows that the classes are reasonably balanced and that combined text length differs across classes, but not enough to act as a standalone predictor.

The evaluation confirms that Random Forest is the strongest model among the considered alternatives. Its confusion matrix shows approximately 94.7% accuracy and a balanced performance across fake and real articles. The k-NN comparison model reaches about 83.5% accuracy and makes more fake-to-real errors. The feature importance analysis further shows that Random Forest uses both lexical terms and engineered features, making it a practical compromise between performance and interpretability.

Future work should include hyperparameter optimization, probability calibration, validation on external datasets, and deeper analysis of false positives and false negatives. A further extension could compare the current TF-IDF pipeline with transformer-based embeddings, while keeping the Random Forest model as a transparent classical baseline.